National higher school of advanced technologies

Academic year: 2026–2027 | first-year master's degree

Machine learning — practical session 1

Python for Data Exploration and Preprocessing

Learning Objectives

By the end of this practical session, students should be able to load and inspect datasets, select features, create training and test sets, explore text data, identify and handle missing values, scale numerical features, and encode categorical variables using pandas, scikit-learn, and seaborn.

Required libraries: pandas, scikit-learn, seaborn, matplotlib. Use the provided files: Iris_Dataset.csv, txt_file.txt, and employees.csv.

Exercise 1 — Exploring a tabular dataset and creating data splits

Dataset: Iris_Dataset.csv

1. Load the CSV file into a pandas DataFrame named D.
2. Inspect the dataset using info(), describe(), shape, columns, and head(). Briefly explain what each output tells you.
3. Select the two features sepal_length and petal_width and store them in a new DataFrame.
4. Display the first 20 observations.
5. Create a copy of the original DataFrame. Explain why a copy can be useful when experimenting with preprocessing.
6. Split the data into training (80%) and test (20%) sets using train_test_split(). Use random_state=42 so that the split is reproducible.
7. If the dataset contains a class label, use stratify=y when appropriate. State which variables are used as input features and which variable is the target.
8. Verify the number and proportion of observations in the training and test sets.

Discussion: Why should the test set be kept separate until the final evaluation? What could happen if information from the test set influences preprocessing or model selection?

Exercise 2 — Exploring text data

Files and dataset: txt_file.txt and scikit-learn's 20 Newsgroups dataset.

1. Open txt_file.txt using an appropriate text encoding (for example, UTF-8) and read the file.
2. Iterate through the file and display each line. Remove only the trailing newline character where needed.
3. Load the 20 Newsgroups dataset using fetch_20newsgroups() from sklearn.datasets. Start with the training subset and use the default categories unless your instructor specifies otherwise.
4. Display the number of documents and the number of labels. Explain the meaning of these dimensions.
5. Display the first document and inspect its associated target label.
6. Display the available class names using target_names, and explain how numeric target values correspond to class names.
7. As an optional extension, compare the document lengths or count the occurrences of a few frequent words.

Note: The first call to fetch_20newsgroups() may download data. If internet access is unavailable, use a locally available copy or ask your instructor for an alternative.

Exercise 3 — Detecting and handling missing values

Dataset: employees.csv

1. Load employees.csv into a DataFrame and inspect its columns, data types, and first few rows.

2. Count the missing values in each column using isna().sum().
3. Visualize missing-value patterns using a seaborn heatmap. Add a title and clear axis labels.
4. Create a copy of the original DataFrame. In the copy, append one synthetic row containing eight fields; set first name to '-' and gender to 'na'. Fill the other fields with suitable missing-value markers, as specified by the dataset structure.
5. Recount missing values. Explain why '-' and 'na' are not automatically treated as missing by pandas.
6. Convert the chosen markers ('-', 'na', and any other agreed placeholders) to np.nan, using a documented and case-consistent rule.
7. Use sklearn.impute.SimpleImputer to handle missing values. Choose a strategy appropriate to each column: for example, most_frequent for categorical columns and median for numerical columns. Do not impute identifiers without justification.
8. Compare the missing-value counts before and after preprocessing, and explain the limitations of imputation.

Important: Fit an imputer on training data only when preparing data for a predictive model, then use the fitted imputer to transform validation and test data. This helps prevent data leakage.

Exercise 4 — Feature scaling, binarization, and categorical encoding

Part A. Scaling and binarization

Consider the following numerical matrix:

Feature 1	Feature 2	Feature 3
1	-1	2
2	0	0
0	1	-1

1. Apply StandardScaler, MinMaxScaler, MaxAbsScaler, RobustScaler, and normalize from scikit-learn to the matrix. Use the appropriate API for each method.
2. Display the transformed matrix for each method and compare the results.
3. Explain the main idea of each method and identify whether it scales each feature or normalizes each sample.
4. Binarize the original matrix using an explicitly stated threshold. Explain how the threshold affects the output.

Discussion: Why might scaling matter for algorithms such as k-nearest neighbors, support vector machines, and gradient-based models? Is scaling always necessary for decision trees?

Part B. Encoding categorical variables

Consider the following dataset:

Feature 1	Feature 2	Feature 3	Target
male	from US	uses Safari	US
female	from Europe	uses Safari	France
female	from Asia	uses Firefox	Japan
male	from Africa	uses Chrome	Algeria

1. Use OrdinalEncoder to convert the three categorical input features into numeric values. Display the categories_ attribute and explain that the assigned numbers are category codes, not necessarily meaningful rankings.
2. Encode the target variable separately using LabelEncoder or OrdinalEncoder, and display the mapping between class names and numeric labels.
3. Apply OneHotEncoder to the target variable and display the resulting binary representation. Explain the difference between integer label encoding and one-hot encoding.
4. Discuss why one-hot encoding is often preferable to arbitrary ordinal codes for nominal input features.

Submission and expected deliverables

- A well-organized Python script or Jupyter notebook containing the code for all four exercises.
- Outputs, tables, and plots required by each exercise.
- Short answers to the discussion questions, using clear technical explanations.
- A brief conclusion identifying which preprocessing operations are appropriate for numerical, categorical, and text data.

Good practice

- Use meaningful variable names and add concise comments.
- Set random_state where reproducibility is required.
- Keep the original data unchanged; perform experiments on copies.
- Check data types and category values before preprocessing.
- Avoid data leakage by fitting preprocessing steps only on training data when building a machine-learning pipeline.